CPP01 ex03 : Unnecessary violence

Le code complet, expliqué ligne par ligne

Le plan pour demain

1. Lis la partie « L'idée à retenir » ci-dessous.
2. Écris les fichiers dans l'ordre de ce document, en lisant les explications au passage. Le code sans explications, plus facile à copier, se trouve en annexe.
3. Lance les vérifications de la partie « Vérifier ».
4. Avant de rendre l'exercice, retire ce PDF du dossier : il ne fait pas partie des fichiers à rendre.

L'idée à retenir

Bob et Jim n'ont pas leur propre arme : ils **partagent** `club`. C'est pour ça que la deuxième attaque affiche le nouveau type. Le choix entre référence et pointeur dépend ensuite d'une seule question : l'humain peut-il être sans arme ?

	HumanA	HumanB
Attribut	`Weapon &weapon;`	`Weapon *weapon;`
Outil	Référence	Pointeur
Peut être sans arme ?	Non	Oui (`NULL`)
Reçoit l'arme	Dans le constructeur	Plus tard, avec `setWeapon()`
Accès à l'arme	`weapon.getType()`	`weapon->getType()`

Si les humains stockaient une simple `Weapon weapon;`, ils garderaient une copie : la deuxième attaque afficherait encore `crude spiked club`, et le test du sujet échouerait.

Le code, ligne par ligne

Les lignes vides ne sont pas commentées.

Weapon.hpp

> Déjà terminé : **ne change rien**, ce fichier est identique au tien.

```
1   #ifndef WEAPON_HPP
```

Protection contre la double inclusion : si `WEAPON_HPP` n'est pas encore défini, le compilateur lit la suite ; sinon, il saute tout jusqu'au `#endif`.

```
2   # define WEAPON_HPP
```

Définit `WEAPON_HPP` : la prochaine inclusion de ce fichier sera ignorée, ce qui évite de déclarer la classe deux fois.

```
4   #include <string>
```

Donne accès à `std::string`, utilisé pour l'attribut et les paramètres.

```
6   class Weapon
```

Déclare la classe `Weapon`.

```
7   {
```

Ouvre le corps de la classe.

```
8       private:
```

Ce qui suit n'est accessible que depuis les fonctions de la classe.

```
9           std::string type;
```

L'attribut : `std::string` est son type et `type` est son nom. On ne peut donc pas écrire `type` à la place d'un type de retour.

```
11      public:
```

Ce qui suit est accessible depuis l'extérieur de la classe.

```
12          Weapon(const std::string &type);
```

Constructeur : même nom que la classe, pas de type de retour. `const std::string &` évite une copie et accepte aussi un texte entre guillemets. Comme c'est le seul constructeur déclaré, `Weapon w;` sans argument ne compile pas.

```
13          const std::string & getType() const;
```

Le type de retour est `const std::string &` : on renvoie l'attribut lui-même, sans copie et en lecture seule. Le `const` final promet que la fonction ne modifie pas l'objet.

```
14          void setType(const std::string &type);
```

`void` car elle ne renvoie rien. Pas de `const` final : cette fonction modifie l'objet.

```
15  };
```

Ferme la classe. Le `;` est obligatoire.

```
17  #endif
```

Ferme le `#ifndef` du début du fichier.

Weapon.cpp

> Remplace le contenu actuel de ton fichier.

```
1    #include "Weapon.hpp"
```

Inclut la déclaration de la classe, sans laquelle le compilateur ne connaît pas `Weapon`.

```
3    Weapon::Weapon(const std::string &type) : type(type)
```

`Weapon::` indique que la fonction appartient à la classe. Après le `:` vient la liste d'initialisation : dans `type(type)`, le premier `type` est l'attribut et celui entre parenthèses est le paramètre.

```
4    {
```

Début du corps du constructeur.

```
5    }
```

Corps vide : tout le travail est fait par la liste d'initialisation.

```
7    const std::string &Weapon::getType() const
```

Reprend la signature du `.hpp`, avec `Weapon::` juste avant le nom. Le `const` final doit être répété ici.

```
8    {
```

Début de la fonction.

```
9        return (this->type);
```

Renvoie l'attribut. Grâce au type de retour, l'appelant reçoit une référence vers lui, pas une copie.

```
10   }
```

Fin de la fonction.

```
12   void Weapon::setType(const std::string &type)
```

Définition de `setType`. Le paramètre porte le même nom que l'attribut, d'où le `this->` plus bas.

```
13   {
```

Début de la fonction.

```
14       this->type = type;
```

`this` pointe sur l'objet courant, donc `this->type` est l'attribut, et `type` seul le paramètre. Écrire `type = type;` ne changerait pas l'attribut.

```
15   }
```

Fin de la fonction.

HumanA.hpp

```
1    #ifndef HUMANA_HPP
```

Protection contre la double inclusion : si `HUMANA_HPP` n'est pas encore défini, le compilateur lit la suite ; sinon, il saute tout jusqu'au `#endif`.

```
2    # define HUMANA_HPP
```

Définit `HUMANA_HPP` : la prochaine inclusion de ce fichier sera ignorée, ce qui évite de déclarer la classe deux fois.

```
4   #include <string>
```

Pour l'attribut `name`.

```
5   #include "Weapon.hpp"
```

Pour que le compilateur connaisse `Weapon`.

```
7   class HumanA
```

Déclare la classe `HumanA`.

```
8   {
```

Ouvre le corps de la classe.

```
9       private:
```

Attributs accessibles seulement depuis la classe.

```
10          std::string name;
```

Le nom de l'humain.

```
11          Weapon      &weapon;
```

Le point clé. Le `&` en fait une référence : HumanA ne possède pas de copie, `weapon` est un autre nom pour l'arme d'origine. Elle ne peut jamais être vide, ce qui garantit que HumanA est toujours armé.

```
13      public:
```

Fonctions accessibles depuis l'extérieur.

```
14          HumanA(const std::string &name, Weapon &weapon);
```

Constructeur : reçoit le nom et l'arme. `Weapon &weapon` n'a pas de `const`, car la référence stockée n'est pas constante et doit désigner le vrai `club`.

```
15          void attack() const;
```

Affiche l'attaque. `const` car elle ne modifie pas l'objet.

```
16  };
```

Ferme la classe.

```
18  #endif
```

Ferme le `#ifndef` du début du fichier.

HumanA.cpp

```
1   #include <iostream>
```

Pour `std::cout` et `std::endl`.

```
2   #include "HumanA.hpp"
```

Déclaration de `HumanA`, qui inclut aussi `Weapon` et `<string>`.

```
4   HumanA::HumanA(const std::string &name, Weapon &weapon) : name(name), weapon(weapon)
```

Une référence doit être liée dès sa création : `weapon(weapon)` doit donc se trouver dans la liste d'initialisation, l'écrire dans le corps serait trop tard. Garde le même ordre que dans le `.hpp` : `name`, puis `weapon` (sinon `-Wall` signale une erreur).

```
5   {
```

Début du corps du constructeur.

```
6   }
```

Corps vide : tout est déjà initialisé.

```
8   void HumanA::attack() const
```

Définition de `attack`, avec `HumanA::` et le `const` final.

```
9   {
```

Début de la fonction.

```
10      std::cout << this->name << " attacks with their " << this->weapon.getType() << std::endl;
```

Une référence s'utilise comme un objet normal, d'où le point : `weapon.getType()`. Comme c'est le vrai `club`, on lit toujours son type actuel. `std::endl` ajoute le retour à la ligne.

```
11  }
```

Fin de la fonction.

HumanB.hpp

```
1   #ifndef HUMANB_HPP
```

Protection contre la double inclusion : si `HUMANB_HPP` n'est pas encore défini, le compilateur lit la suite ; sinon, il saute tout jusqu'au `#endif`.

```
2   # define HUMANB_HPP
```

Définit `HUMANB_HPP` : la prochaine inclusion de ce fichier sera ignorée, ce qui évite de déclarer la classe deux fois.

```
4   #include <string>
```

Pour l'attribut `name`.

```
5   #include "Weapon.hpp"
```

Pour que le compilateur connaisse `Weapon`.

```
7   class HumanB
```

Déclare la classe `HumanB`.

```
8   {
```

Ouvre le corps de la classe.

```
9       private:
```

Attributs accessibles seulement depuis la classe.

```
10          std::string name;
```

Le nom de l'humain.

```
11          Weapon      *weapon;
```

Le point clé. Le `*` en fait un pointeur : il peut ne pointer vers rien (`NULL`), puis recevoir une arme plus tard. C'est ce qu'il faut à HumanB, qui n'a pas toujours d'arme.

```
13      public:
```

Fonctions accessibles depuis l'extérieur.

```
14          HumanB(const std::string &name);
```

Constructeur : seulement le nom, puisque HumanB ne reçoit pas d'arme à sa création.

```
15          void setWeapon(Weapon &weapon);
```

Donne une arme. `Weapon &` reçoit le vrai `club`, sans copie ; le `main` peut donc écrire `jim.setWeapon(club)` sans `&`.

```
16          void attack() const;
```

Affiche l'attaque. `const` car elle ne modifie pas l'objet.

```
17  };
```

Ferme la classe.

```
19  #endif
```

Ferme le `#ifndef` du début du fichier.

HumanB.cpp

```
1   #include <cstddef>
```

Définit `NULL`.

```
2   #include <iostream>
```

Pour `std::cout` et `std::endl`.

```
3   #include "HumanB.hpp"
```

Déclaration de `HumanB`.

```
5   HumanB::HumanB(const std::string &name) : name(name), weapon(NULL)
```

`weapon(NULL)` indique qu'il n'y a pas encore d'arme. Sans ça, le pointeur contiendrait une adresse aléatoire.

```
6   {
```

Début du corps du constructeur.

```
7   }
```

Corps vide : tout est déjà initialisé.

```
9   void HumanB::setWeapon(Weapon &weapon)
```

Définition de `setWeapon`. Avec `Weapon weapon` (sans `&`), on travaillerait sur une copie détruite à la fin de la fonction.

```
10  {
```

Début de la fonction.

```
11      this->weapon = &weapon;
```

`&weapon` donne l'adresse du vrai `club`, stockée dans l'attribut. Le pointeur reste donc valide tant que `club` existe, et il voit les changements de `setType()`. `this->` distingue l'attribut du paramètre.

```
12  }
```

Fin de la fonction.

```
14  void HumanB::attack() const
```

Définition de `attack`.

```
15  {
```

Début de la fonction.

```
16      if (this->weapon == NULL)
```

Vérifie si HumanB est armé. Suivre un pointeur `NULL` risquerait de faire planter le programme.

```
17      {
```

Début du cas « sans arme ».

```
18          std::cout << this->name << " has no weapon" << std::endl;
```

Message affiché quand il n'y a pas d'arme. Le sujet n'impose pas ce texte, tu peux le changer.

```
19          return ;
```

Quitte la fonction : la ligne d'attaque ne s'exécute pas.

```
20      }
```

Fin du cas « sans arme ».

```
21      std::cout << this->name << " attacks with their " << this->weapon->getType() << std::endl;
```

Ici le pointeur est valide. On utilise `->` et non `.` car `weapon` est un pointeur : on suit l'adresse, puis on appelle `getType()` sur l'arme.

```
22  }
```

Fin de la fonction.

main.cpp

> Ajoute seulement les **lignes 4 et 5** : le reste est déjà dans ton fichier.

```
1   #include <string>
```

Inclusion déjà présente dans ton fichier.

```
2   #include <iostream>
```

Inclusion déjà présente dans ton fichier.

```
3   #include "Weapon.hpp"
```

Déclaration de `Weapon`.

```
4   #include "HumanA.hpp"
```

À ajouter. Sans cette ligne : erreur `'HumanA' was not declared`.

```
5   #include "HumanB.hpp"
```

À ajouter. Sans cette ligne : erreur `'HumanB' was not declared`.

```
7   int main()
```

Point d'entrée du programme.

```
8   {
```

Début de `main`.

```
9       {
```

Bloc du test HumanA : ce qui y est créé est détruit au `}` de la ligne 15.

```
10          Weapon club = Weapon("crude spiked club");
```

Crée l'arme. Le texte devient un `std::string` temporaire, accepté grâce au `const std::string &` du constructeur.

```
11          HumanA bob("Bob", club);
```

Crée Bob. Sa référence `weapon` est liée à `club`.

```
12          bob.attack();
```

Affiche `Bob attacks with their crude spiked club`.

```
13          club.setType("some other type of club");
```

Modifie `club`, sans toucher à `bob`.

```
14          bob.attack();
```

Affiche `Bob attacks with their some other type of club` : Bob utilise bien le même `club`.

```
15      }
```

Fin du bloc : `bob` et `club` sont détruits.

```
16      {
```

Bloc du test HumanB.

```
17          Weapon club = Weapon("crude spiked club");
```

Crée une nouvelle arme, indépendante de la première.

```
18          HumanB jim("Jim");
```

Crée Jim, sans arme : son pointeur vaut `NULL`.

```
19          jim.setWeapon(club);
```

Le pointeur de Jim reçoit l'adresse de `club`.

```
20          jim.attack();
```

Affiche `Jim attacks with their crude spiked club`.

```
21          club.setType("some other type of club");
```

Modifie `club`, sans toucher à `jim`.

```
22          jim.attack();
```

Affiche `Jim attacks with their some other type of club`.

```
23      }
```

Fin du bloc : `jim` et `club` sont détruits.

```
24      return 0;
```

Fin normale du programme.

```
25  }
```

Fin de `main`.

Makefile

> Attention : les lignes 13, 16, 19 et 22 doivent commencer par une vraie tabulation, sinon `make` affiche `missing separator`.

```
1   NAME = violence
```

Nom de l'exécutable. Le sujet n'en impose pas : tu peux le changer.

```
3   CXX = c++
```

Le compilateur C++.

```
4   CXXFLAGS = -Wall -Wextra -Werror -std=c++98
```

Active les avertissements, les transforme en erreurs et impose la norme C++98.

```
6   SRCS = main.cpp Weapon.cpp HumanA.cpp HumanB.cpp
```

Les fichiers sources.

```
7   OBJS = $(SRCS:.cpp=.o)
```

Remplace `.cpp` par `.o` dans `SRCS` pour obtenir la liste des fichiers objets.

```
8   HDRS = Weapon.hpp HumanA.hpp HumanB.hpp
```

Les en-têtes, utilisés pour la règle de la ligne 15.

```
10  all: $(NAME)
```

`make` seul exécute la première règle : elle construit l'exécutable.

```
12  $(NAME): $(OBJS)
```

L'exécutable dépend de tous les `.o`.

```
13      $(CXX) $(CXXFLAGS) $(OBJS) -o $(NAME)
```

Assemble les `.o` en exécutable. **Commence par une vraie tabulation.**

```
15  %.o: %.cpp $(HDRS)
```

Règle générique : chaque `.o` dépend de son `.cpp` et des en-têtes, et il est donc recompilé si l'un d'eux change.

```
16      $(CXX) $(CXXFLAGS) -c $< -o $@
```

Compile un seul fichier. `$<` est le `.cpp`, `$@` est le `.o`. **Commence par une vraie tabulation.**

```
18  clean:
```

`make clean` :

```
19      rm -f $(OBJS)
```

supprime les `.o`. **Vraie tabulation.**

```
21  fclean: clean
```

`make fclean` lance d'abord `clean`, puis :

```
22      rm -f $(NAME)
```

supprime l'exécutable. **Vraie tabulation.**

```
24  re: fclean all
```

`make re` : tout supprimer, puis tout recompiler.

```
26  .PHONY: all clean fclean re
```

Indique que ces noms sont des commandes, même si un fichier porte le même nom.

Vérifier

Une fois tous les fichiers écrits, lance cette commande dans le dossier `ex03` :

```
make && ./violence && valgrind --leak-check=full ./violence
```

Le programme doit afficher ces quatre lignes :

```
Bob attacks with their crude spiked club
Bob attacks with their some other type of club
Jim attacks with their crude spiked club
Jim attacks with their some other type of club
```

`valgrind` doit indiquer `All heap blocks were freed -- no leaks are possible` et `ERROR SUMMARY: 0 errors`.
Ce code a été testé ainsi le 4 novembre 2026, avec `-Wall -Wextra -Werror -std=c++98`.

Si ça ne marche pas

- `missing separator` : une ligne de commande du `Makefile` commence par des espaces, remplace-les par une tabulation.
- `'HumanA' was not declared` : il manque les lignes 4 et 5 de `main.cpp`.
- `will be initialized after` : l'ordre de la liste d'initialisation diffère de celui des attributs dans le `.hpp`.
- `uninitialized reference member` : `weapon(weapon)` manque dans la liste d'initialisation de `HumanA`.

Annexe : le code sans explications

À copier fichier par fichier. Après un collage, vérifie les tabulations du Makefile et l'indentation.

Weapon.hpp

```
#ifndef WEAPON_HPP
# define WEAPON_HPP

#include <string>

class Weapon
{
    private:
        std::string type;

    public:
        Weapon(const std::string &type);
        const std::string & getType() const;
        void setType(const std::string &type);
};

#endif
```

Weapon.cpp

```
#include "Weapon.hpp"

Weapon::Weapon(const std::string &type) : type(type)
{
}

const std::string &Weapon::getType() const
{
    return (this->type);
}

void Weapon::setType(const std::string &type)
{
    this->type = type;
}
```

HumanA.hpp

```
#ifndef HUMANA_HPP
# define HUMANA_HPP

#include <string>
#include "Weapon.hpp"

class HumanA
{
    private:
        std::string name;
        Weapon      &weapon;

    public:
        HumanA(const std::string &name, Weapon &weapon);
        void attack() const;
};

#endif
```

HumanA.cpp

```
#include <iostream>
#include "HumanA.hpp"

HumanA::HumanA(const std::string &name, Weapon &weapon) : name(name), weapon(weapon)
{
}

void HumanA::attack() const
{
    std::cout << this->name << " attacks with their " << this->weapon.getType() << std::endl;
}
```

HumanB.hpp

```
#ifndef HUMANB_HPP
# define HUMANB_HPP

#include <string>
#include "Weapon.hpp"

class HumanB
{
    private:
        std::string name;
        Weapon      *weapon;

    public:
        HumanB(const std::string &name);
        void setWeapon(Weapon &weapon);
        void attack() const;
};

#endif
```

HumanB.cpp

```
#include <cstddef>
#include <iostream>
#include "HumanB.hpp"

HumanB::HumanB(const std::string &name) : name(name), weapon(NULL)
{
}

void HumanB::setWeapon(Weapon &weapon)
{
    this->weapon = &weapon;
}

void HumanB::attack() const
{
    if (this->weapon == NULL)
    {
        std::cout << this->name << " has no weapon" << std::endl;
        return ;
    }
    std::cout << this->name << " attacks with their " << this->weapon->getType() << std::endl;
}
```

main.cpp

```
#include <string>
#include <iostream>
#include "Weapon.hpp"
#include "HumanA.hpp"
#include "HumanB.hpp"

int main()
{
    {
        Weapon club = Weapon("crude spiked club");
        HumanA bob("Bob", club);
        bob.attack();
        club.setType("some other type of club");
        bob.attack();
    }
    {
        Weapon club = Weapon("crude spiked club");
        HumanB jim("Jim");
        jim.setWeapon(club);
        jim.attack();
        club.setType("some other type of club");
        jim.attack();
    }
    return 0;
}
```

Makefile

```
NAME = violence

CXX = c++
CXXFLAGS = -Wall -Wextra -Werror -std=c++98

SRCS = main.cpp Weapon.cpp HumanA.cpp HumanB.cpp
OBJS = $(SRCS:.cpp=.o)
HDRS = Weapon.hpp HumanA.hpp HumanB.hpp

all: $(NAME)

$(NAME): $(OBJS)
    $(CXX) $(CXXFLAGS) $(OBJS) -o $(NAME)

%.o: %.cpp $(HDRS)
    $(CXX) $(CXXFLAGS) -c $< -o $@

clean:
    rm -f $(OBJS)

fclean: clean
    rm -f $(NAME)

re: fclean all

.PHONY: all clean fclean re
```